

Matplotlib for Signals

A complete plotting guide: one signal, many signals, subplots, and discrete stems

Everything you need to visualise continuous & discrete-time signals in an exam.

1. The Two Ways to Plot (read this first)

Matplotlib has **two styles**. Knowing which you're using prevents 90% of confusion.

pyplot style (quick, one plot)

You call `plt.something()` and it draws on the “current” figure. Great for a single quick plot.

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(-4, 4, 800)
x = np.sin(2*np.pi*0.5*t)

plt.figure()                # start a new figure
plt.plot(t, x, label="x(t)")
plt.title("A simple signal")
plt.xlabel("t"); plt.ylabel("Amplitude")
plt.grid(True)
plt.legend()
plt.show()                  # display window(s)
```

object style (needed for subplots)

You get a `fig` and one or more `ax` (axes) objects and call methods on them. **You must use this style the moment you want more than one subplot.**

```
fig, ax = plt.subplots()    # one figure, one axes
ax.plot(t, x, label="x(t)")
ax.set_title("A simple signal")
ax.set_xlabel("t"); ax.set_ylabel("Amplitude")
ax.grid(True)
ax.legend()
plt.show()
```

Naming map (pyplot → axes method): `plt.title` → `ax.set_title`, `plt.xlabel` → `ax.set_xlabel`, `plt.ylabel` → `ax.set_ylabel`, `plt.xlim` → `ax.set_xlim`. `plot`, `grid`, `legend`, `stem` keep the same name. When in doubt, use the object style — it always works.

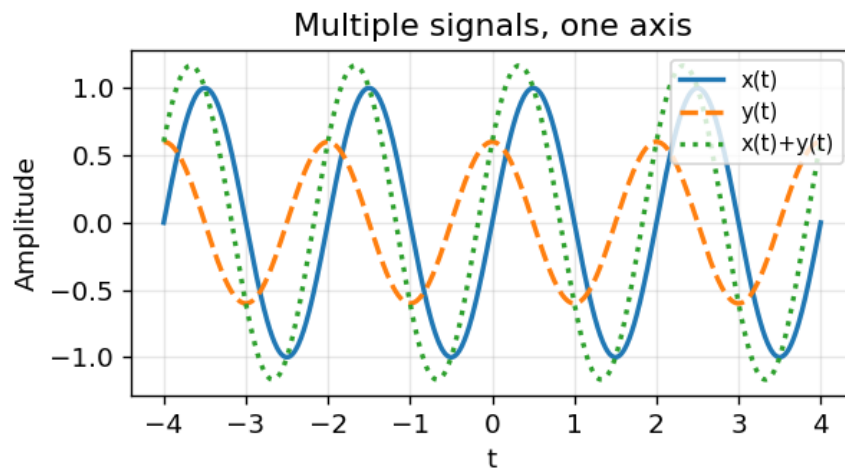
2. Multiple Signals on the SAME Graph

Just call `plot` more than once before showing. Each gets its own colour automatically. **Always pass `label=...` and call `legend()`** so the grader can tell them apart.

```
t = np.linspace(-4, 4, 800)
x = np.sin(2*np.pi*0.5*t)
y = 0.6*np.cos(2*np.pi*0.5*t)

plt.figure()
plt.plot(t, x, label="x(t)")
plt.plot(t, y, label="y(t)")
plt.plot(t, x + y, label="x(t) + y(t)")
plt.title("Three signals on one axis")
plt.xlabel("t"); plt.ylabel("Amplitude")
```

```
plt.grid(True)
plt.legend()
plt.show()
```



This is the pattern for “plot $x(t)$ and $x(-t)$ on the same figure” or “show x , x_e , x_o together” — exactly what the even-odd and time-reversal questions ask for.

3. Multiple Signals on SEPARATE Graphs (subplots)

Use `plt.subplots(nrows, ncols)`. It returns the figure and an array of axes.

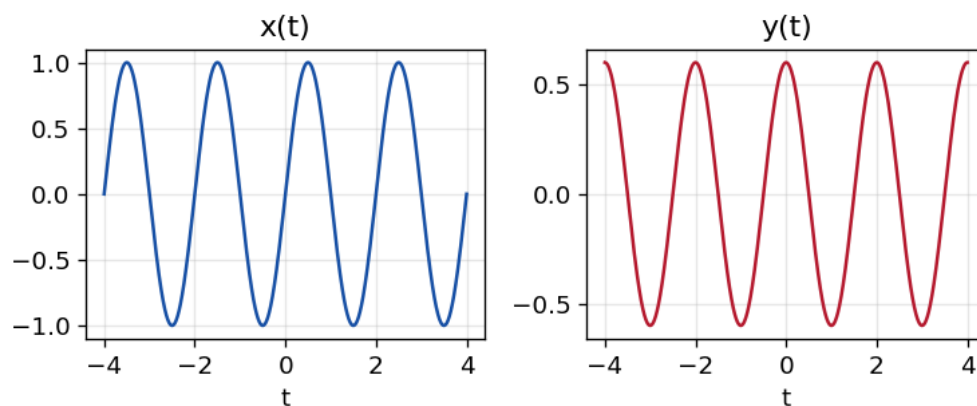
Two side by side (1 row, 2 columns)

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(9, 4))

ax1.plot(t, x); ax1.set_title("x(t)")
ax1.set_xlabel("t"); ax1.set_ylabel("Amplitude"); ax1.grid(True)

ax2.plot(t, y); ax2.set_title("y(t)")
ax2.set_xlabel("t"); ax2.set_ylabel("Amplitude"); ax2.grid(True)

fig.tight_layout() # stops labels overlapping
plt.show()
```



Four in a grid (2 rows, 2 columns)

When you have a 2-D grid, `axes` is a 2-D array. Loop with `axes.flat` to style them all.

```
fig, axes = plt.subplots(2, 2, figsize=(8, 6))
```

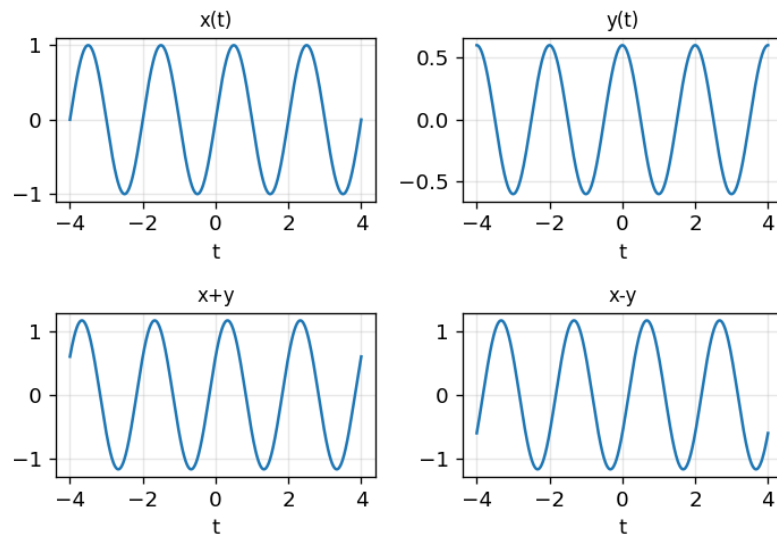
```

axes[0, 0].plot(t, x);    axes[0, 0].set_title("x(t)")
axes[0, 1].plot(t, y);    axes[0, 1].set_title("y(t)")
axes[1, 0].plot(t, x + y); axes[1, 0].set_title("x + y")
axes[1, 1].plot(t, x - y); axes[1, 1].set_title("x - y")

for ax in axes.flat:      # apply to every subplot
    ax.grid(True)
    ax.set_xlabel("t"); ax.set_ylabel("Amplitude")

fig.tight_layout()
plt.show()

```



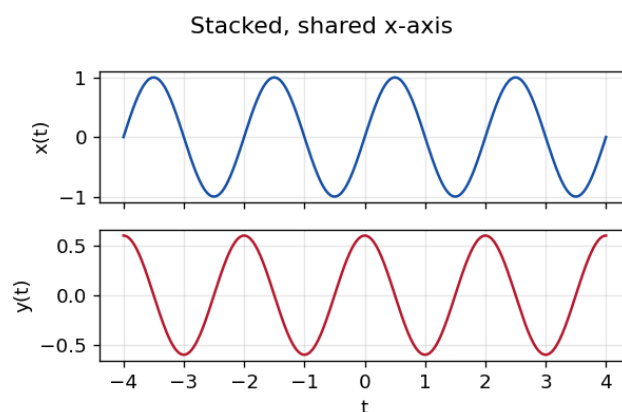
Stacked & sharing an axis

For a signal and its transform, stacking vertically with a shared x-axis reads nicely.

```

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 5), sharex=True)
ax1.plot(t, x); ax1.set_ylabel("x(t)"); ax1.grid(True)
ax2.plot(t, y); ax2.set_ylabel("y(t)"); ax2.set_xlabel("t"); ax2.grid(True)
fig.suptitle("Stacked, shared x-axis")
fig.tight_layout()
plt.show()

```



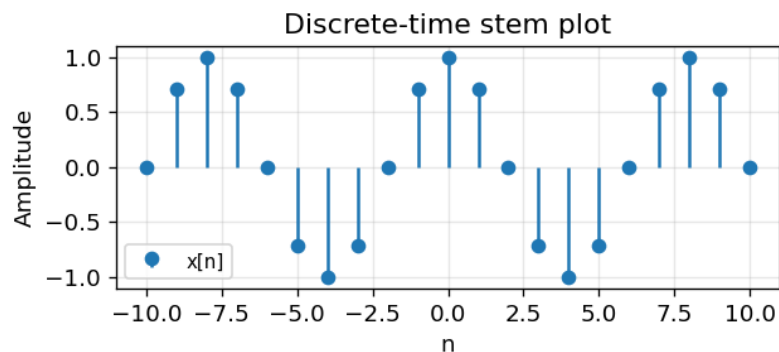
Indexing axes: `subplots(1,2)` → `axes[0]`, `axes[1]` (1-D). `subplots(2,2)` → `axes[0,0]` ... `axes[1,1]` (2-D). Unpacking like `fig, (ax1, ax2) = ...` only works when you know the exact count.

4. Discrete-Time Signals: Stem Plots

For $x[n]$ (integer index) use `stem`, not `plot` — it draws a vertical line + marker at each sample, which is the correct representation of a discrete signal.

```
n = np.arange(-10, 11)
xn = np.cos(np.pi/4 * n)

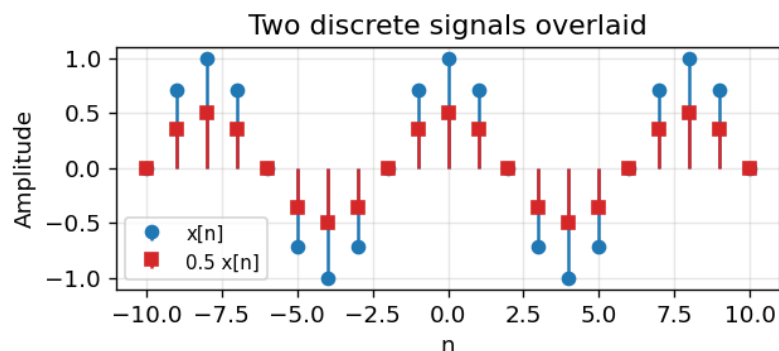
fig, ax = plt.subplots(figsize=(7, 3))
markerline, stemlines, baseline = ax.stem(n, xn, label="x[n]")
baseline.set_visible(False)      # hide the horizontal base line
ax.grid(True, alpha=0.3)
ax.set_xlabel("n"); ax.set_ylabel("Amplitude")
ax.legend()
plt.show()
```



`stem` returns three things: the markers, the vertical lines, and the baseline. Capturing them lets you hide the baseline (cleaner) or recolour.

Styling a stem & overlaying two

```
fig, ax = plt.subplots(figsize=(7, 3))
ax.stem(n, xn, linefmt="C0-", markerfmt="C0o", basefmt=" ", label="x[n]")
ax.stem(n, 0.5*xn, linefmt="C3-", markerfmt="C3s", basefmt=" ", label="0.5 x[n]")
ax.set_xlabel("n"); ax.set_ylabel("Amplitude")
ax.grid(True, alpha=0.3); ax.legend()
plt.show()
```



`basefmt=" "` (a space) hides the baseline without needing the return values. `linefmt` / `markerfmt` take colour+style codes: `C0`, `C1`, ... are the default colour cycle; markers include `o` circle, `s` square, `^` triangle, `D` diamond.

5. Styling: Colours, Lines, Markers

Inside plot()

```
plt.plot(t, x, color="red",      linestyle="--", linewidth=2, label="dashed")
plt.plot(t, y, color="#1750A5", linestyle="-.", linewidth=1.5, label="dashdot")
plt.plot(t, x, "g:", label="green dotted (shorthand)")
plt.plot(t, y, "b-o", label="blue line + circle markers")
```

Reference values

Colours: name ("red" , "green" , "black"), single letter (r g b c m y k), hex ("#1750A5"), or cycle ("C0" ...).

Line styles: "-" solid, "--" dashed, ":" dotted, "-." dash-dot.

Markers: o circle, . point, s square, ^ triangle, * star, x cross, D diamond.

Shorthand combines them: "r-", "g:o", "k-.".

6. Axes, Labels, Limits, Legends

```
# pyplot style
plt.title("My title", fontsize=14, fontweight="bold")
plt.xlabel("t"); plt.ylabel("Amplitude")
plt.xlim(-4, 4); plt.ylim(-1.5, 1.5)      # zoom / fix range
plt.xticks(np.arange(-4, 5, 1))           # tick positions
plt.grid(True, alpha=0.3)                 # faint grid
plt.legend(loc="upper right", fontsize=9, framealpha=0.9)

# reference lines through the origin (handy for signals)
plt.axhline(0, color="black", linewidth=0.8)
plt.axvline(0, color="black", linewidth=0.8)
```

legend locations: "best" (auto), "upper right", "upper left", "lower right", "lower left", "center".

On the object side these are `ax.set_title`, `ax.set_xlim`, `ax.set_xticks`, `ax.axhline`, etc.

7. Extras That Score Marks

Shade a region / area under a curve

```
plt.plot(t, x, label="x(t)")
plt.fill_between(t, x, 0, alpha=0.2)      # shade between x and 0
```

Annotate a point

```
peak_t = t[np.argmax(x)]
plt.annotate("peak", xy=(peak_t, x.max()), xytext=(peak_t, 1.3),
            arrowprops=dict(arrowstyle="->"))
```

Figure size & saving

```
plt.figure(figsize=(8, 4))                # width, height in inches
# ... plotting ...
plt.savefig("signal.png", dpi=150, bbox_inches="tight") # instead of show()
```

8. Copy-Paste Templates

Continuous: original vs transformed on one axis

```
def plot_pair(t, x, y, title):
    plt.figure()
```

```
plt.plot(t, x, label="x(t)")
plt.plot(t, y, label="y(t)")
plt.title(title)
plt.xlabel("t"); plt.ylabel("Amplitude")
plt.grid(True); plt.legend()
plt.show()
```

Three signals (e.g. x, even, odd)

```
def plot_three(t, x, xe, xo):
    plt.figure()
    plt.plot(t, x, label="x(t)")
    plt.plot(t, xe, label="xe(t)")
    plt.plot(t, xo, label="xo(t)")
    plt.title("Even-Odd Decomposition")
    plt.xlabel("t"); plt.ylabel("Amplitude")
    plt.grid(True); plt.legend()
    plt.show()
```

Discrete: reusable stem helper

```
def stem_plot(ax, n, x, label):
    markerline, stemlines, baseline = ax.stem(n, x, label=label)
    baseline.set_visible(False)
    ax.grid(True, alpha=0.3)
    ax.set_xlabel("n"); ax.set_ylabel("Amplitude")

# usage:
fig, ax = plt.subplots(figsize=(9, 4))
stem_plot(ax, n, xn, "original x[n]")
stem_plot(ax, n, xn_shift, "shifted")
ax.legend(); fig.tight_layout(); plt.show()
```

9. Common Errors & Fixes

- **Nothing appears:** you forgot `plt.show()` (or, when saving, use `plt.savefig` *before* `show`).
- **All plots pile onto one figure:** call `plt.figure()` before each new independent plot, or use separate `fig, ax`.
- **Legend is empty / missing:** you must pass `label="..."` in every `plot` / `stem`, and call `legend()`.
- **Labels overlap between subplots:** add `fig.tight_layout()` at the end.
- **x and y must have same first dimension:** `t` and `x` have different lengths — check `len(t)==len(x)`.
- **Using `plt.set_title` (crashes):** pyplot uses `plt.title`; only *axes* use `ax.set_title`.
- **stem baseline clutter:** `baseline.set_visible(False)` or `basefmt=" "`.
- **Jagged “curve”:** too few points — use `np.linspace(a, b, 800)` or more for continuous signals.

Pair this with the NumPy cheat sheet. NumPy builds the signal; Matplotlib shows it.